Data Partitioning

Data partitioning is process of dividing a large database (DB) into smaller, more manageable parts called partitions or shards. Each partition is independent and contains a subset of the overall data.

In data partitioning, the dataset is typically partitioned based on a certain criterion, such as data range, data size, or data type. Each partition is then assigned to a separate processing node, which can perform operations on its assigned data subset independently of the others.

Data partitioning can help improve performance and scalability of large-scale data processing applications, as it allows processing to be distributed across multiple nodes, minimizing data transfer and reducing processing time. Secondly, by distributing the data across multiple nodes or servers, the workload can be balanced, and the system can handle more requests and process data more efficiently.

Data partitioning can be done in several ways, including horizontal partitioning, vertical partitioning, and hybrid partitioning.

1. Partitioning Methods

Designing an effective partitioning scheme can be challenging and requires careful consideration of the application requirements and the characteristics of the data being processed. Below are three of the most popular schemes used by various large-scale applications.

a. Horizontal Partitioning: Also known as sharding, horizontal data partitioning involves dividing a database table into multiple partitions or shards, with each partition containing a subset of rows. Each shard is typically assigned to a different database server, which allows for parallel processing and faster query execution times.

For example, consider a social media platform that stores user data in a database table. The platform might partition the user table horizontally based on the geographic location of the users, so that users in the United States are stored in one shard, users in Europe are stored in another shard, and so on. This way, when a user logs in and their data needs to be accessed, the query can be directed to the appropriate shard, minimizing the amount of data that needs to be scanned.

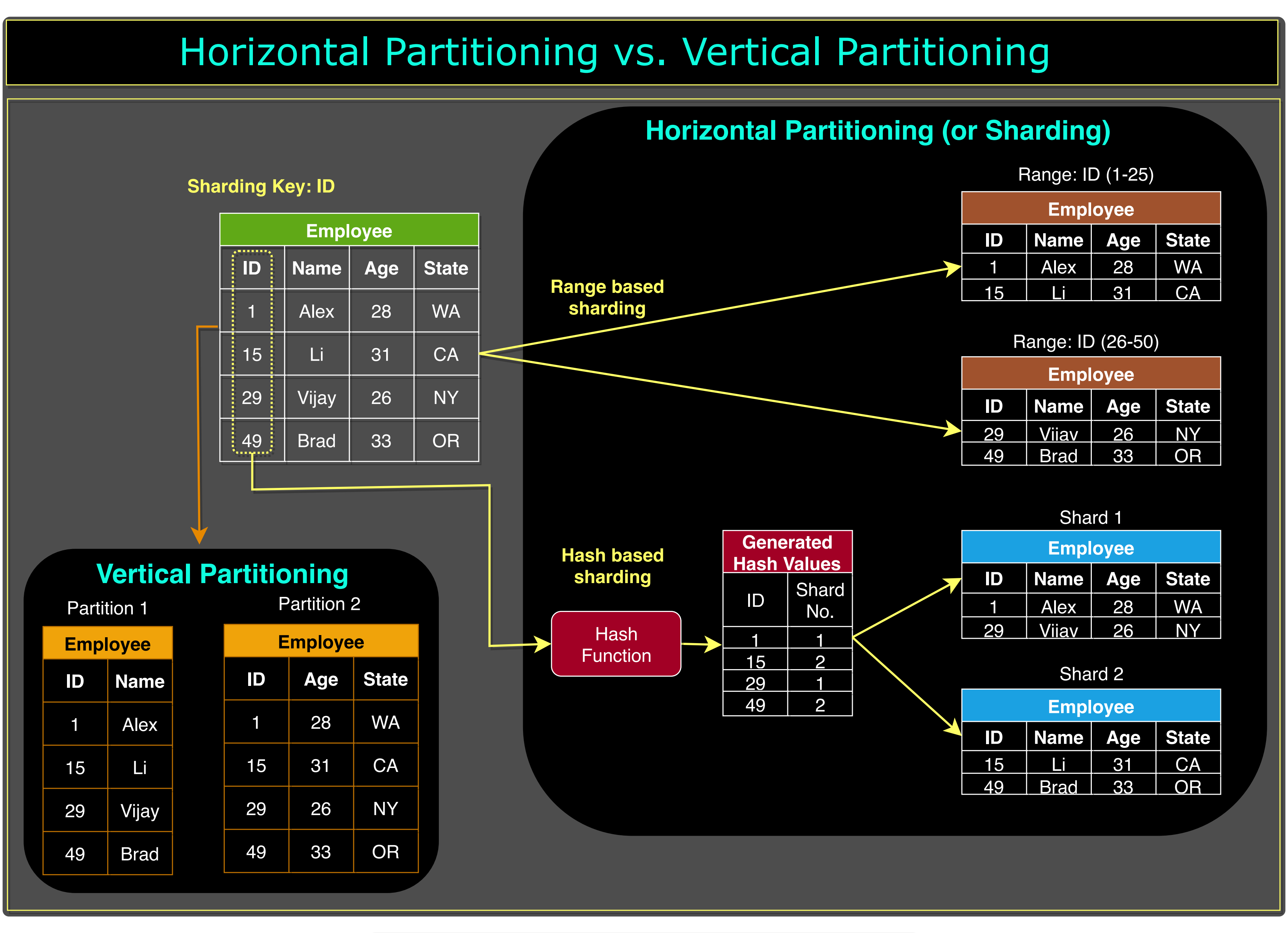
The key problem with this approach is that if the value whose range is used for partitioning isn't chosen carefully, then the partitioning scheme will lead to unbalanced servers. For instance, partitioning users based on their geographic location assumes an even distribution of users across different regions, which may not be valid due to the presence of densely or sparsely populated areas.

b. Vertical Partitioning: Vertical data partitioning involves splitting a database table into multiple partitions or shards, with each partition containing a subset of columns. This technique can help optimize performance by reducing the amount of data that needs to be scanned, especially when certain columns are accessed more frequently than others.

For example, consider an e-commerce website that stores customer data in a database table. The website might partition the customer table vertically based on the type of data, so that personal information such as name and address are stored in one shard, while order history and payment information are stored in another shard. This way, when a customer logs in and their order history needs to be accessed, the query can be directed to the appropriate shard, minimizing the amount of data that needs to be scanned.

c. Hybrid Partitioning: Hybrid data partitioning combines both horizontal and vertical partitioning techniques to partition data into multiple shards. This technique can help optimize performance by distributing the data evenly across multiple servers, while also minimizing the amount of data that needs to be scanned.

For example, consider a large e-commerce website that stores customer data in a database table. The website might partition the customer table horizontally based on the geographic location of the customers, and then partition each shard vertically based on the type of data. This way, when a customer logs in and their data needs to be accessed, the query can be directed to the appropriate shard, minimizing the amount of data that needs to be scanned. Additionally, each shard can be stored on a different database server, allowing for parallel processing and faster query execution times.



Horizontal Partitioning vs. Vertical Partitioning

2. Partitioning Criteria

Data partitioning criteria are the factors or characteristics of data that can be used to divide a large dataset into smaller parts or partitions. Here are some of the most common criteria used for data partitioning:

a. Key or Hash-based Partitioning: Under this scheme, we apply a hash function to some key attributes of the entity we are storing; that yields the partition number. For example, if we have 100 DB servers and our ID is a numeric value that gets incremented by one each time a new record is inserted. In this example, the hash function could be 'ID % 100', which will give us the server number where we can store/read that record. This approach should ensure a uniform allocation of data among servers. The fundamental problem with this approach is that it effectively fixes the total number of DB servers, since adding new servers means changing the hash function which would require redistribution of data and downtime for the service. A workaround for this problem is to use 'Consistent Hashing'.

b. List partitioning: In this scheme, each partition is assigned a list of values, so whenever we want to insert a new record, we will see which partition contains our key and then store it there. For example, we can decide all users living in Iceland, Norway, Sweden, Finland, or Denmark will be stored in a partition for the Nordic countries.

c. Round-robin partitioning: This is a very simple strategy that ensures uniform data distribution. With 'n' partitions, the 'i' tuple is assigned to partition (i mod n).

d. Composite Partitioning: Under this scheme, we combine any of the above partitioning schemes to devise a new scheme. For example, first applying a list partitioning scheme and then a hash-based partitioning. Consistent hashing could be considered a composite of hash and list partitioning where the hash reduces the key-space to a size that can be listed.

3. Common Problems of Data Partitioning

On a partitioned database, there are certain extra constraints on the different operations that can be performed. Most of these constraints are due to the fact that operations across multiple tables or multiple rows in the same table will no longer run on the same server. Below are some of the constraints and additional complexities introduced by Partitioning:

a. Joins and Denormalization: Performing joins on a database that is running on one server is straightforward, but once a database is partitioned and spread across multiple machines it is often not feasible to perform joins that span database partitions. Such joins will not be performance efficient since data has to be compiled from multiple servers. A common workaround for this problem is to denormalize the database so that queries that previously required joins can be performed from a single table. Of course, the service now has to deal with denormalization's perils, such as data inconsistency.

b. Referential integrity: As we saw that performing a cross-partition query on a partitioned database is not feasible; similarly, trying to enforce data integrity constraints such as foreign keys in a partitioned database can be extremely difficult.

Most RDBMS do not support foreign keys constraints across databases on different database servers. This means, applications that require referential integrity on partitioned databases often have to enforce it in application code. Often in such cases, applications have to run regular SQL jobs to clean up dangling references.

c. Rebalancing: There could be many reasons we have to change our partitioning scheme:

1. The data distribution is not uniform, e.g., there are a lot of places for a particular ZIP code that cannot fit into one database partition.
2. There is a lot of load on a partition, e.g., there are too many requests being handled by the DB partition dedicated to user photos.

In such cases, either we have to create more DB partitions or have to rebalance existing partitions, which means the partitioning scheme changed and all existing data moved to new locations. Doing this without incurring downtime is extremely difficult. Using a scheme like directory-based Partitioning does make rebalancing a more palatable experience at the cost of increasing the complexity of the system and creating a new single point of failure (i.e. the lookup service/database).

← Previous
Caching

Next →
Indexes

☐ Mark as Completed